PHDS Website: Developer Guide

Planetary Health and Development Stewards, first production iteration

- 1. Purpose of this guide
- 2. What the application is
- 3. Technology stack
- 4. Repository layout
- 5. Running the application locally
  - 5.1 Prerequisites
  - 5.2 First-time setup
  - 5.3 Development loop
  - 5.4 Making the first administrator
- 6. Configuration reference
  - 6.1 Core
  - 6.2 Seeded accounts
  - 6.3 Public site
  - 6.4 M-Pesa (Daraja)
  - 6.5 Website assistant (Gemini)
  - 6.6 Team chat and real-time
  - 6.7 Mail
- 7. Data model
  - 7.1 Users and accounts
  - 7.2 Tables
  - 7.3 Enumerations
- 8. Roles and permissions
- 9. Feature specifications
  - 9.1 Public site
  - 9.2 Contact form and newsletter
  - 9.3 Blog
  - 9.4 Donations
  - 9.5 Team chat
  - 9.6 End-to-end encryption (team chat)
  - 9.7 Website assistant (chatbot)
  - 9.8 Authentication and accounts
- 10. Route reference
  - 10.1 Public
  - 10.2 Staff and administrators
  - 10.3 Team chat
- 11. Scheduled jobs and background work
- 12. Deployment
  - 12.1 The container
  - 12.2 Render (or any container host)
  - 12.3 Known deployment gaps
  - 12.4 Reverse proxy and HTTPS
- 13. Operations runbook
  - 13.1 Useful commands
  - 13.2 Troubleshooting donations
  - 13.3 Troubleshooting chat
  - 13.4 Troubleshooting the assistant
  - 13.5 Changing copy and content
- 14. Testing
- 15. Known issues and gaps
- 16. Roadmap
- 17. Glossary

- 18. Handover checklist

1. Purpose of this guide

This guide is for the next developer who takes over the PHDS website. It explains what the application does, how it is put together, how to run it, how to deploy it, and what is not finished. It describes the code as it stood at commit `a9e875b` (merge of `mike-linux`, October 2026). If the code has moved on, check the repository history against this document before relying on a detail.

The project is a volunteer effort. Keep this guide, the repository and the handover notes together, and update the guide whenever behaviour changes.

Where this guide says something is **not done** or **not verified**, treat it as a known gap. Section 15 lists every known gap in one place.

2. What the application is

PHDS is a public website for a Kenyan public benefit organisation working on the Lake Victoria basin. It has five parts:

1. **Public site.** Home, About, Board, What We Do, six focus area pages, Where We Work, Get Involved, Contact, and a public blog.
2. **Donations.** A general donation and campaign donations, paid through M-Pesa (Safaricom Daraja, STK push).
3. **Back office.** A staff and administrator area for writing blog posts, managing campaigns and users, and viewing donations.
4. **Team chat.** Direct messages and groups between staff, with end-to-end encryption, replies, forwarding and notifications.
5. **Website assistant.** A chatbot on the public site, answering only from site content, powered by Google Gemini.

3. Technology stack

Layer	Technology	Version or note
Language (server)	PHP	8.3 or newer required by `composer.json`. The Dockerfile uses PHP 8.4.
Framework	Laravel	13.x
Page bridge	Inertia.js server adapter	`inertiajs/inertia-laravel` 2.x
Client framework	React	19
Client bridge	`@inertiajs/react`	2.x (must match the server adapter)
Styling	Tailwind CSS	4.x, with `@tailwindcss/forms`
Build	Vite	8.x, `laravel-vite-plugin`
Real-time	Laravel Reverb (server), `laravel-echo` and `pusher-js` (client)	Optional. Chat falls back to polling.
Icons	`lucide-react`	1.x
Language (client)	TypeScript	Type checks with `npx tsc --noEmit`
Database	SQLite by default, PostgreSQL supported	The Docker image includes `pdo_pgsql`
Container	Docker, Alpine Linux, nginx, php-fpm, supervisord	See section 12

Both the PHP and the JavaScript dependencies are pinned by `composer.lock` and `package-lock.json`. Commit both.

4. Repository layout

```
app/
  Enums/                Fixed value lists stored as strings (UserRole, DonationStatus, ...)
  Http/Controllers/     Request handlers. Admin/ holds back-office controllers.
  Http/Requests/        Form validation (BlogRequest, DonationRequest, UserRequest, ...)
  Http/Middleware/      HandleInertiaRequests (shared props), EnsureNoUsersExist
  Models/               Eloquent models
  Policies/             Authorisation rules for chat, users, campaigns, donations
  Services/Chat/        Chat notifications, presentation, cleanup, realtime helper
  Services/Chatbot/     Gemini client, site knowledge, scope gate
  Services/Donations/   Donation state machine and idempotency
  Services/Mpesa/       Daraja client and its error types
  Console/Commands/     user:role, donations:reconcile
  Events/               Broadcast events for chat and notifications
  Notifications/        Database notification for chat activity
config/
  phds.php              All public site copy, board, focus areas and seed accounts
  mpesa.php             M-Pesa settings and limits
  chatbot.php           Gemini settings and limits
  chat.php              Chat limits and realtime client settings
  broadcasting.php      Broadcast driver (null or reverb)
  reverb.php            Reverb server settings
database/
  migrations/           Schema, all keys are UUIDs except bookkeeping tables
  seeders/              UserSeeder (from .env), BlogSeeder (sample posts)
docker/                 nginx.conf and supervisord.conf
docs/                   This guide, MPESA_SANDBOX.md
resources/js/
  app.tsx               Inertia entry point and page resolver
  Pages/                One React component per Inertia page
  Layouts/              site-layout (public), admin-layout (back office), auth layout
  Components/           Shared UI, chat/, donate/
  lib/                  routes, e2ee (encryption), chat-crypto, key-store, sound, echo
routes/
  web.php               All web routes (see section 10)
  auth.php              Breeze authentication routes
  channels.php          Broadcast channel authorisation
  console.php           Scheduled jobs
tests/
  Feature/              Feature tests (161 passing at this commit)
  Unit/                 Unit tests
  Concerns/             Shared test helpers
```

5. Running the application locally

5.1 Prerequisites

- PHP 8.3 or newer with the extensions `bcmath curl dom fileinfo intl mbstring openssl pdo_sqlite sqlite3 xml zip`
- Composer 2
- Node.js 20 or newer, and npm
- Optional: ngrok or cloudflared, needed only to receive M-Pesa callbacks locally

5.2 First-time setup

```
git clone https://github.com/Mikelesnr/phds-site.git
cd phds-site
composer install
npm install
cp .env.example .env
php artisan key:generate
touch database/database.sqlite
php artisan migrate --seed
npm run build
php artisan serve
```

Then open `http://127.0.0.1:8000`.

`migrate --seed` creates the sample blog posts and the accounts defined in `.env` (section 7.1). Set `SEED_ADMIN_EMAIL`, `SEED_ADMIN_PASSWORD` and `SEED_STAFF_PASSWORD` before seeding, or the user seeder refuses to create the admin.

5.3 Development loop

- `npm run dev` serves the front end with hot reload while `php artisan serve` runs in another terminal.

- `npx tsc --noEmit` type checks the front end.
- `php artisan test` runs the test suite. Tests do not need `npm run build` (the test base class disables Vite).
- `php artisan route:list` lists every route.

5.4 Making the first administrator

The registration page only works while there are no users. In a fresh database, either:

- open `/register` and create the first account (it becomes an administrator), or
- run `php artisan user:role someone@example.org admin` for an existing account.

Once any user exists, `/register` returns 404. Other accounts are created by an administrator at `/admin/users`.

6. Configuration reference

All settings come from `.env`. Never commit `.env`. The table lists every variable the application reads. Variables marked *optional* have a working default.

6.1 Core

Variable	Purpose	Notes
`APP_NAME`	Application name	`PHDS`
`APP_ENV`	`local` or `production`	Production disables debug output
`APP_KEY`	Encryption key for sessions and cookies	Generated by `key:generate`. Must be stable in production.
`APP_DEBUG`	Show detailed errors	Must be `false` in production
`APP_URL`	Public address of the site	Used to build links and the M-Pesa callback fallback
`DB_CONNECTION`	`sqlite` or `pgsql`	Production should use PostgreSQL
`SESSION_DRIVER`	Session storage	`database` in `.env.example`
`CACHE_STORE`	Cache storage	`database` in `.env.example`. Holds the M-Pesa access token and rate limits.
`QUEUE_CONNECTION`	Queue driver	`database`. No queue worker is currently required.
`LOG_CHANNEL`, `LOG_LEVEL`	Logging	Daraja refusals log at `error`. Use `stderr` in containers.

6.2 Seeded accounts

Variable	Purpose
`SEED_ADMIN_NAME`	Display name of the seeded administrator
`SEED_ADMIN_EMAIL`	Email of the seeded administrator. Required to seed.
`SEED_ADMIN_PASSWORD`	Password of the seeded administrator. Required to seed.
`SEED_STAFF_1_NAME`, `SEED_STAFF_1_EMAIL`	First staff account (optional)
`SEED_STAFF_2_NAME`, `SEED_STAFF_2_EMAIL`	Second staff account (optional)
`SEED_STAFF_PASSWORD`	Shared password for both staff accounts. If empty, staff are skipped.

The seeder treats `.env` as the source of truth. Running it again resets name, role and password for those accounts. See section 7.1.

6.3 Public site

Variable	Purpose
`PHDS_CONTACT_EMAIL`	Public contact email (optional). Hidden until set.
`PHDS_CONTACT_LOCATION`	Location shown in the footer. Default `Lake Victoria Basin, Kenya`.

6.4 M-Pesa (Daraja)

Variable	Purpose	Notes
`MPESA_ENV`	`sandbox` or `production`	Selects the Safaricom host
`MPESA_BASE_URL`	Overrides the host	Used only for testing against a stand-in
`MPESA_CONSUMER_KEY`, `MPESA_CONSUMER_SECRET`	OAuth credentials from the Daraja app	Secret. Never log or share.
`MPESA_SHORTCODE`	Business shortcode	Sandbox test shortcode is `174379`
`MPESA_PASSKEY`	Passkey for the password field	Sandbox value comes from the Daraja simulator page
`MPESA_TRANSACTION_TYPE`	`CustomerPayBillOnline` or `CustomerBuyGoodsOnline`	Paybill or till
`MPESA_PARTY_B`	Receiving party. Till number for tills.	Optional. Defaults to the shortcode.
`MPESA_ACCOUNT_REFERENCE`	Text shown on the phone prompt, 12 characters max	Default `PHDS`
`MPESA_CALLBACK_TOKEN`	Secret path segment in the callback URL	Required. Must be a long random value.
`MPESA_CALLBACK_BASE`	Public HTTPS address Safaricom can reach	Required. Changes with every ngrok session.
`MPESA_CALLBACK_IPS`	Optional comma separated allow list of caller addresses	Leave empty in sandbox
`DONATION_MIN_AMOUNT`, `DONATION_MAX_AMOUNT`	Limits in whole shillings	Defaults 10 and 150000

6.5 Website assistant (Gemini)

Variable	Purpose	Default
`GEMINI_API_KEY`	Server-side key. Never sent to the browser.	none
`GEMINI_MODEL`	Model name	`gemini-2.5-flash`
`CHATBOT_ENABLED`	Switch the assistant on or off	`true`
`CHATBOT_MAX_OUTPUT_TOKENS`	Length cap per answer	`450`
`CHATBOT_PER_MINUTE`	Messages per minute per visitor	`8`
`CHATBOT_PER_IP_DAILY`	Messages per day per address	`40`
`CHATBOT_DAILY_LIMIT`	Messages per day across all visitors	`500`

6.6 Team chat and real-time

Variable	Purpose
`BROADCAST_CONNECTION`	`null` (polling only) or `reverb`
`CHAT_POLL_SECONDS`	Polling interval when websockets are off. Default `6`.
`REVERB_APP_ID`, `REVERB_APP_KEY`, `REVERB_APP_SECRET`	Reverb application credentials. Generate the key and secret as random strings.
`REVERB_HOST`, `REVERB_PORT`, `REVERB_SCHEME`	Address the browser connects to, for example `yourdomain.org`, `443`, `https`
`REVERB_PUBLISH_HOST`, `REVERB_PUBLISH_PORT`, `REVERB_PUBLISH_SCHEME`	Address the app publishes to, usually the local Reverb server

6.7 Mail

Mail uses the Laravel defaults (`MAIL_MAILER=log` in `.env.example`). Password reset and email verification mails are written to the log until a real mailer is configured. Contact form submissions are stored in the database and do not send email (section 15).

7. Data model

7.1 Users and accounts

The `users` table holds a UUID `id`, `name`, `email`, `password`, `role` (a string, validated by the `UserRole` enum), and `email_verified_at`. Roles are:

Role	Meaning
`admin`	Manages users, campaigns, donations, featured posts and all blog posts
`staff`	Writes and edits blog posts, and uses team chat

`UserSeeder` creates one administrator and up to two staff accounts from `.env`. It matches users by email and, on each run, resets their name, role and password to the `.env` values. It checks that the stored hash matches before finishing. Because of this, `.env` is the source of truth for seeded accounts. Change a seeded password in `.env` and reseed, rather than changing it in the database.

7.2 Tables

Table	Purpose	Key columns
`users`	Accounts	`id` (uuid), `role`, `email`
`blogs`	Posts for the public blog and campaign stories	`id`, `user_id`, `title`, `slug`, `body`, `images` (JSON), `videos` (JSON), `is_featured`, `published_at`
`inquiries`	Contact form and newsletter sign-ups	`type` (`contact` or `newsletter`), `name`, `email`, `interest`, `message`
`campaigns`	Donation campaigns	`id`, `title`, `slug`, `summary`, `goal_amount` (KES), `blog_id`, `is_active`, `starts_at`, `ends_at`
`donations`	One row per donation attempt	`id` (uuid from the browser), `campaign_id`, `amount`, `phone`, `status`, `request_fingerprint`, `checkout_request_id`, `mpesa_receipt`, `result_code`, `result_desc`, `paid_amount`, `paid_at`, `expires_at`, `last_checked_at`
`mpesa_callbacks`	Every callback received from Safaricom, kept as received	`donation_id`, `checkout_request_id`, `result_code`, `outcome`, `payload`
`conversations`	Direct chats and groups	`id`, `type`, `visibility`, `name`, `direct_key`, `created_by`, `key_version`
`conversation_user`	Membership, role and read marker	`conversation_id`, `user_id`, `role`, `last_read_at`
`messages`	Chat messages (encrypted)	`id`, `conversation_id`, `user_id`, `sender_id`, `seq`, `ciphertext`, `iv`, `key_version`, `edited_at`, `deleted_at`
`user_keys`	Each person’s chat identity	`user_id`, `public_key`, `encrypted_private_key`
`conversation_keys`	A conversation key sealed for one member	`conversation_id`, `user_id`, `key_version`, `wrapped_key`, `granted_by`
`notifications`	Chat notifications	`notifiable` (uuid morph), `type`, `data`, `read_at`
`cache`, `jobs`, `sessions`	Laravel infrastructure	Standard

All primary keys are UUIDs (`HasUuids`). Messages also carry a per-conversation `seq` number, which gives the thread its order. Random UUIDs cannot be used for order, so `seq` is calculated from the highest existing value in the conversation.

7.3 Enumerations

Enum	Values
`UserRole`	`admin`, `staff`
`ConversationType`	`direct`, `group`

`ConversationVisibility`	`open`, `private`
`MemberRole`	`moderator`, `member`
`DonationStatus`	`pending`, `awaiting_payment`, `succeeded`, `failed`, `cancelled`, `expired`

Enums are stored as plain strings. The database does not enforce them, so the enum classes are the guard. Add new values to the enum and to the validation rules together.

8. Roles and permissions

Action	Public	Staff	Admin
View public pages, blogs, donate pages	Yes	Yes	Yes
Make a donation	Yes	Yes	Yes
Use the back office and chat	No	Yes	Yes
Create and edit blog posts	No	Yes	Yes
Edit any blog post	No	Yes (not enforced, see section 15)	Yes
Delete a blog post	No	No	Yes
Set featured posts	No	No	Yes
Manage campaigns	No	No	Yes
View donations and export CSV	No	No	Yes
Manage users	No	No	Yes
Administer any chat it is not in	No	No	No (no special access)

Chat has its own roles that do not depend on the user role: the **moderator** of a group (its creator, or a successor) can manage that group, and **members** cannot. Admins have no special access to chats.

Authorisation lives in three places. Controllers call `verify()` (for blogs), `Gate::authorize()` with policies (for users, campaigns, donations, chats and messages), or check `$user->isAdmin()`. Policies are in `app/Policies`.

9. Feature specifications

9.1 Public site

Pages are rendered by `PageController` from the content in `config/phds.php`. Editing copy, stats, board members, focus areas, counties and partnership routes is done there, not in React components. The public site has a sticky header with a mega menu (About, What We Do, Where We Work, Get Involved), a footer with the newsletter form, a bottom banner, and the chat assistant. Light and dark modes are both supported.

Focus area pages come from `config/phds.php` by slug. There are six: `blue-economy`, `climate-change`, `green-economy`, `global-health`, `one-health` and `research-and-policy`.

9.2 Contact form and newsletter

Submissions are validated and stored in `inquiries`. A honeypot field silently discards most bot submissions. Newsletter sign-ups are stored once per email address. **Nothing sends email yet**: submissions are only in the database, so someone has to check that table, or a notification needs adding (section 15).

9.3 Blog

- **Public:** `/blogs` lists published posts. Each post opens in a popup from the list. There is no separate public URL per post. Scheduled posts (future `published_at`) are hidden.
- **Back office:** `/admin/blogs` lists posts; `/admin/blogs/create` and `/admin/blogs/{id}/edit` create and edit them. Staff and admins can use these pages.
- **Pictures:** a picture is either an `https://` link to an image hosted elsewhere, or a path to a picture shipped with the site (for example `/images/weighing-catch.jpg`). Other schemes, protocol-relative links and `..` are rejected.
- **Videos:** YouTube and Vimeo links only. `App\Support\VideoEmbed` converts them to safe embed addresses. Other hosts

are rejected.

- **Featured:** an administrator can mark posts as featured. The home page shows up to `Blog::HOME_FEATURED_LIMIT` (currently 3), newest first.
- **Delete:** administrators only.
- **Campaign stories:** a campaign can attach a published blog post as its story (section 9.5).

9.4 Donations

9.4.1 What a donor can do

A donor opens `/donate` (general) or `/donate/campaigns/{slug}` (campaign). They choose an amount, enter a Safaricom number and optionally a name, and press Donate. Their phone receives an M-Pesa prompt, which they approve with their PIN. The page shows “Check your phone” and then the result.

9.4.2 Donation statuses

Status	Meaning	Donor message
`pending`	Saved, M-Pesa not yet asked	Starting your payment
`awaiting_payment`	Prompt requested, waiting for the PIN	Check your phone
`succeeded`	Paid. Counted in campaign totals.	Thank you, with the receipt
`failed`	M-Pesa refused, or another error	Not completed, no charge
`cancelled`	Donor cancelled on the phone (result 1032)	Cancelled, no charge
`expired`	No answer in time (result 1037 or timeout)	Timed out, no charge

`succeeded`, `failed`, `cancelled` and `expired` are terminal. `DonationStatus::isTerminal()` checks this.

9.4.3 Idempotency

The donation id is a UUID generated in the donor’s browser when they first press Donate. It is kept in session storage and sent with every retry. The server guarantees:

- A repeated request with the same id returns the existing donation and never asks M-Pesa again.
- A request with the same id but different amount, phone or campaign is refused with 409.
- A second donation cannot start on the same phone while one is still waiting (default expiry 3 minutes). The response includes the waiting donation’s id, and the page follows it.
- The M-Pesa request is made once, claimed by an atomic update on `initiated_at`. Two simultaneous requests cannot both ask for money.
- If the server saved a donation but crashed before asking M-Pesa, the next retry asks M-Pesa for it.
- A callback that arrives twice is recorded twice in `mpesa_callbacks` but counted once.
- A receipt number cannot be attached to two donations.

9.4.4 The M-Pesa flow

1. The browser posts `{id, amount, phone, campaign_id, donor_name}` to `POST /donate`.
2. `DonationRequest` validates the input and normalises the phone to `2547XXXXXXXX`.
3. `DonationService::start` checks idempotency, the per-phone limit (5 per hour) and the waiting-payment rule, then saves the donation as `pending`.
4. `MpesaClient::stkPush` gets an OAuth token (cached), builds the Daraja request (shortcode, timestamp, password, callback URL), and posts it.
5. If Daraja returns `ResponseCode 0` with a `CheckoutRequestID`, the donation becomes `awaiting_payment` and stores the ids. The `CheckoutRequestID` links the later callback to this donation.
6. Daraja refuses: the donation becomes `failed`, and the full refusal is logged as `M-Pesa STK push refused` with Daraja’s body.
7. Daraja times out or returns a gateway error: the donation stays `awaiting_payment` and is settled later, because we cannot tell whether the prompt was sent.
8. The browser polls `GET /donate/status/{id}` every 3 seconds. After 40 seconds the status endpoint asks Daraja directly (`stkpushquery`), at most once every 20 seconds.
9. Daraja posts the result to `POST /mpesa/callback/{token}/{donation-id}`. The token is in the path, so only someone who knows it can post. `DonationService::applyCallback` records the callback, then applies it inside a transaction with a row

lock.

10. `donations:reconcile` (every minute, section 11) settles donations still open after their expiry window.

Result codes handled: `0` succeeded; `1032` cancelled; `1037` expired; any other code failed. A late success still counts if the donor approved after the donation expired.

9.4.5 The amount that counts

Campaign totals use `paid_amount` when present, otherwise `amount`. Only `succeeded` donations count. `Campaign::raised()` sums them.

9.4.6 Campaigns

Campaigns have a title, a unique slug, a summary, an optional goal, an optional attached blog post, an active switch and optional start and end dates. A campaign is open for donations when it is active and within its dates. Inactive campaigns return 404 to the public. A campaign that has donations cannot be deleted; switch it off instead.

Only published blog posts can be attached. Campaign pages show the goal progress and the attached story.

9.4.7 Admin screens

`/admin/campaigns` creates and edits campaigns. `/admin/donations` lists donations with filters (status, campaign, search) and exports the current filter to CSV (`/admin/donations/export`). The totals shown are successful donations only.

9.4.8 Known limits of the donation flow

See section 15. The most important: a successful sandbox payment has not yet been completed end to end.

9.5 Team chat

9.5.1 Conversations

- **Direct messages:** exactly one per pair of people. `Conversation::directBetween` creates it on first use, keyed by the two ids in order.
- **Open groups:** anyone signed in can join. They appear in “Open groups to join” in the sidebar. Names must be unique among open groups (case-insensitive).
- **Private groups:** visible only to members. Non-members get 404. Administrators get no special access.
- **Moderator:** the creator of a group. The moderator can rename the group, change its description, add and remove members, delete any message in the group and delete the group. If a moderator is deleted from the system, the longest-standing member takes over.

9.5.2 Messages

- **Send:** the browser encrypts the text, then posts ciphertext, IV and key version with a client-chosen UUID. Retrying the same message returns the original, never a duplicate.
- **Order:** by `seq` within the conversation. All messages of a conversation load at once with the page.
- **Edit:** only the author, and only their own messages, re-encrypted and marked “edited”.
- **Delete:** the author, or the moderator of the group. The ciphertext and IV are wiped from the database, and the bubble shows “This message was deleted”.
- **Reply:** the reply reference lives inside the encrypted message body, so the server cannot see which message is being replied to. The quoted excerpt is decrypted in the browser.
- **Forward:** the browser decrypts a message and re-encrypts it for each chosen conversation. Forwards keep the original author in the envelope.
- **Read state:** each membership stores `last_read_at`. Unread counts drive the sidebar and the navigation badge.

9.5.3 Notifications and sound

- Notifications are stored in the `notifications` table and shown in a bell in the header.
- New messages and additions to groups create notifications. Several unread messages in one conversation collapse into one notification with a count.
- Real-time delivery uses Reverb when enabled. Otherwise the bell polls every `CHAT_POLL_SECONDS`.
- Sounds use the browser’s Web Audio API, so no audio files are needed. Browsers require a click or key press before playing sound. A mute toggle is stored in local storage.

9.6 End-to-end encryption (team chat)

This is the most security-sensitive part of the application. Read it fully before changing anything in `resources/js/lib/e2ee.ts`, `chat-crypto.ts` or `key-store.ts`.

9.6.1 Key hierarchy

Key	Where it lives	Notes
Identity key pair (ECDH P-256)	Public key on the server. Private key is locked with the person's chat passphrase (PBKDF2-SHA256, 600,000 iterations, AES-GCM) and stored on the server as a blob. An unlocked copy is kept in IndexedDB as a non-extractable CryptoKey.	Passphrase is never sent
Conversation key (AES-256-GCM)	Sealed separately for each member with their public key (ephemeral ECDH, HKDF-SHA256, AES-GCM). Stored in `conversation_keys`.	Versioned by `key_version`
Message encryption	AES-256-GCM with the conversation key and a fresh random 96-bit IV per message. Additional authenticated data binds conversation id, message id, sender id and key version.	Moving or replaying a ciphertext fails authentication

9.6.2 What the server holds

The server stores public keys, the locked private key blob, sealed conversation keys, ciphertext, IVs and metadata (who, when, which conversation). It cannot read messages, cannot unlock private keys, and cannot derive conversation keys. Deleted messages have their ciphertext wiped.

9.6.3 Key rotation

When a member leaves, is removed, or is deleted from the system, the conversation's `key_version` increases. The departed member's sealed keys are deleted. Later messages use the new version, which the departed member cannot open. Remaining members keep access to older versions.

9.6.4 Sharing keys

A conversation key is started by the people allowed to start it: either person in a direct chat, or the moderator of a group. The starter must include their own copy. Afterwards, anyone holding the key can seal it for a member who lacks it, subject to these rules: anyone in a direct chat, anyone in an open group, and only the moderator in a private group.

9.6.5 Security codes

Each person has a short security code derived from their public key. The Encryption dialog shows the codes for everyone in the conversation. The application remembers each code and warns when a code changes, which usually means a member reset their identity. Users are asked to compare codes out of band.

9.6.6 Passphrase and recovery

Losing the chat passphrase means losing access to previously stored private keys. There is no recovery. The reset flow creates a new identity, so the person can use the chat again, but messages from before the reset stay unreadable until another member shares the keys again. Users are warned about this in the UI.

9.6.7 Limits of the design

- The encryption protects message content from the server and from anyone with database access. It does not protect against a compromised browser, a malicious member, or screenshots.
- Metadata is visible to the server: who talks to whom, when, and how much.
- The design has not had an independent security review. Commission one before the chat handles sensitive material.

9.7 Website assistant (chatbot)

The assistant is on every public page. It answers only from site content: the organisation, its work, the board, the blog, and a site map of permitted links. Its behaviour is controlled in four layers:

1. **Scope gate** (`ScopeGate`): a cheap check before any model call. It refuses prompt-injection phrases and questions with no meaningful overlap with the site vocabulary, so off-topic questions cost nothing.
2. **System prompt** (`SiteKnowledge::systemPrompt`): instructs the model to answer only from the site knowledge, to refuse other topics, and to never invent facts, contacts or figures.
3. **Output limits:** 450 tokens by default, 6 turns of history, 500 characters per message, thinking disabled on Gemini 2.5 Flash.
4. **Spending limits:** messages per minute per visitor, per day per address, and per day overall (section 6.5). When the daily total is reached, the assistant stops answering and points to the contact page.

Knowledge is cached for 5 minutes and is built only from public content and published posts. Drafts, users, donations and chat data are never included.

Gemini is called from the server with the key in the `x-goog-api-key` header. The key never reaches the browser.

9.8 Authentication and accounts

- Login, logout, password reset and email verification use Laravel Breeze routes (`routes/auth.php`), restyled to the site.
- Login ignores the case of the email address. Breeze's built-in login throttle is the only brute-force protection.
- `/register` is available only while the users table is empty (`EnsureNoUsersExist`). The account it creates is an administrator.
- Passwords follow Laravel's default rules. Users can change their own name, email and password at `/profile`, and can delete their own account there.
- Administrators create, edit, change roles of, and remove other users at `/admin/users`. Nobody can change their own role or delete themselves, so the site always keeps an administrator. Removing a user keeps their blog posts and chat messages, with their name shown as "Former member".

10. Route reference

Routes are listed by area. Run `php artisan route:list` for the full list including method names.

10.1 Public

Method	Path	Purpose
GET	`/`	Home
GET	`/about` , `/about/board`	About, board
GET	`/what-we-do`	Objectives and focus areas
GET	`/focus-areas/{slug}`	Focus area page
GET	`/where-we-work`	Basin and counties
GET	`/get-involved`	Partnership routes and form
GET, POST	`/contact`	Contact page and submission
POST	`/subscribe`	Newsletter sign-up
GET	`/blogs`	Blog list
GET	`/donate`	Donation page (general and open campaigns)
GET	`/donate/campaigns/{slug}`	Campaign page
POST	`/donate`	Start a donation (throttled)
GET	`/donate/status/{donation}`	Donation status (polled)
POST	`/chat`	Website assistant question (throttled)
GET	`/chat/news`	Latest posts for the assistant
POST	`/mpesa/callback/{token}/{donation}`	Daraja callback (no CSRF token, secret in path)

10.2 Staff and administrators

Method	Path	Who	Purpose
GET	`/dashboard`	Staff, admin	Summary and featured posts
GET	`/admin/blogs`	Staff, admin	Blog list
GET, POST	`/admin/blogs/create`, `/admin/blogs`	Staff, admin	Create a post
GET	`/admin/blogs/{blog}/edit`	Staff, admin	Edit page (ownership not enforced)
PUT	`/admin/blogs/{blog}`	Staff, admin	Update (ownership not enforced)
DELETE	`/admin/blogs/{blog}`	Admin	Delete
PATCH	`/admin/blogs/{blog}/featured`	Admin	Toggle featured
GET	`/admin/campaigns` and related	Admin	Campaign management
GET	`/admin/donations`, `/admin/donations/export`	Admin	Donations and CSV export
GET	`/admin/users` and related	Admin	User management
GET	`/profile`	Signed in	Own profile

10.3 Team chat

Method	Path	Purpose
GET	`/team-chat`	Chat home
POST	`/team-chat/direct`	Open a direct chat
POST	`/team-chat/groups`	Create a group
GET	`/team-chat/{conversation}`	Open a conversation
PATCH, DELETE	`/team-chat/{conversation}`	Update or delete a group (moderator)
POST	`/team-chat/{conversation}/join`	Join an open group
POST	`/team-chat/{conversation}/leave`	Leave a group
POST	`/team-chat/{conversation}/members`	Add members (moderator)
DELETE	`/team-chat/{conversation}/members/{user}`	Remove a member (moderator)
POST	`/team-chat/{conversation}/messages`	Send an encrypted message (throttled)
PATCH, DELETE	`/team-chat/messages/{message}`	Edit or delete a message
GET, POST	`/team-chat/{conversation}/keys`	Read or share conversation keys
POST	`/team-chat/{conversation}/read`	Mark read
GET, POST	`/chat-keys/me`, `/chat-keys`, `/chat-keys/reset`	Identity keys
GET	`/notifications`	Notifications for the bell
POST	`/notifications/{id}/read`, `/notifications/read-all`	Mark read

11. Scheduled jobs and background work

Job	Schedule	Purpose
`donations:reconcile`	Every minute, without overlap	Asks Daraja about donations still open, and expires those past their window

The scheduler must actually run. Locally use `php artisan schedule:work`. In production, either run `schedule:work` as a process or add a cron entry:

```
* * * * * cd /var/www/html && php artisan schedule:run >> /dev/null 2>&1
```

The current Docker image does not run the scheduler (section 12.3). Until that is added, donations whose callback

never arrives stay open. The status page still asks Daraja on each poll, which covers donors who keep the page open.

Other background work runs inline: notifications, chat broadcasts and mail are sent when the request happens. The queue driver is `database`, but no queue worker is required by the current code.

12. Deployment

12.1 The container

The `Dockerfile` has three build stages:

1. **composer_base:** installs PHP dependencies with `php:8.4-cli-alpine`.
2. **frontend:** builds the front end with `node:20-alpine`.
3. **backend:** `php:8.4-fpm-alpine` with nginx and supervisord, the application code, and the built assets.

The container exposes port 80. Its command is `supervisord`, which starts three processes:

Program	Purpose
`php-fpm`	PHP processing
`nginx`	HTTP server (`docker/nginx.conf`). Proxies `/app/` and `/apps/` to Reverb on port 8080.
`reverb`	Websocket server. Runs only when `BROADCAST_CONNECTION=reverb`, otherwise it sleeps.

12.2 Render (or any container host)

1. Set every variable from section 6 that applies, at minimum `APP_KEY`, `APP_URL`, `APP_ENV=production`, `APP_DEBUG=false`, the database settings, and the seed variables.
2. Run migrations and seeding on deploy: `php artisan migrate --force` then, once, `php artisan db:seed --force`. The image does not do this automatically.
3. Give the service a persistent database. The default SQLite file lives inside the container and is lost on every redeploy on free plans.
4. Set `MPESA_CALLBACK_BASE` to the public HTTPS address of the service (for example `https://phds.example.org`). Set `MPESA_CALLBACK_TOKEN` to a long random value.
5. For real-time chat, set `BROADCAST_CONNECTION=reverb` and the `REVERB_*` variables. Otherwise the chat polls.

12.3 Known deployment gaps

- **No scheduler.** Add `schedule:run` (section 11) to the container or the host.
- **No automatic migration.** The start command does not run `migrate`, so a new deploy with a new migration will fail until someone runs it.
- **No `render.yaml` in the repository.** An earlier Blueprint was produced separately and is not committed.
- **Entrypoint script not in repository.** An earlier entrypoint that sets the port and runs migrations was produced separately and is not committed. Decide whether to adopt it.

12.4 Reverse proxy and HTTPS

M-Pesa requires the callback address to be HTTPS and publicly reachable. Terminate TLS at the host or proxy and forward `X-Forwarded-*` headers. Laravel must be told to trust the proxy's forwarded headers, or generated links and the M-Pesa callback address can come out as `http`. The repository does not yet contain this setting (see section 15, item 17). Add `$middleware->trustProxies(at: '*')` in `bootstrap/app.php`, restricted to the proxy's address if known. Restrict `MPESA_CALLBACK_IPS` to Safaricom's addresses in production once confirmed.

13. Operations runbook

13.1 Useful commands

Task	Command
Make an existing account an administrator	`php artisan user:role email@example.org admin`

Run one reconcile pass by hand	`php artisan donations:reconcile`
List routes	`php artisan route:list`
Clear cached config after editing `.env`	`php artisan config:clear`
Check a config group	`php artisan config:show mpesa`
Inspect the latest donations	`php artisan tinker --execute="App\Models\Donation::latest('created_at')->take(5)->get(['id','status','result_code','result_desc','created_at'])->toArray();"`
Inspect callbacks	`php artisan tinker --execute="App\Models\MpesaCallback::latest()->take(5)->get(['donation_id','result_code','outcome','created_at'])->toArray();"`

13.2 Troubleshooting donations

Symptom	Where to look	Likely cause
“We could not start the M-Pesa payment”	`grep "M-Pesa STK push refused" storage/logs/laravel.log`	Daraja refused the request. The logged body names the reason, for example `Invalid CallBackURL`.
“Payment request timed out” with result 1037	`mpesa_callbacks` and `result_code`	The prompt could not reach the handset. In sandbox this is normal for test numbers.
Stuck on “Check your phone”	`donations` status and `last_checked_at`	Callback not received. Check the tunnel or domain and whether the scheduler runs.
“A payment is already waiting on this number”	The donation with `existing_id`	Another donation to that number is still open. Wait for it to expire, about 3 minutes.
“Too many attempts”	Rate limiter	Wait a minute. The per-hour per-number limit is 5.
Callback returns 404	Path and token	`MPESA_CALLBACK_TOKEN` does not match the one in the callback URL. Run `config:clear` after changing it.
Donation shows `Unable to lock subscriber` in logs	Daraja error `500.001.1001`	A previous request to that number is still open. Wait at least one minute.

Daraja error reference used by the code is in the Safaricom documentation. `500.001.1001` with “Wrong credentials” means the password does not match the shortcode, passkey and timestamp, so check the passkey and shortcode pair.

13.3 Troubleshooting chat

- **Messages do not update live:** check `BROADCAST_CONNECTION` and the Reverb host settings. The page still polls, so messages appear within `CHAT_POLL_SECONDS`.
- **“Waiting for the encryption key”:** the message was sent before the reader received their key. It arrives once a member with the key opens the conversation.
- **A user cannot read old messages after a reset:** expected. See section 9.6.6.

13.4 Troubleshooting the assistant

- **“not available right now”:** `GEMINI_API_KEY` is missing or `CHATBOT_ENABLED` is false.
- **“reached its limit for today”:** the daily total is reached. Raise `CHATBOT_DAILY_LIMIT` or wait until midnight UTC.

13.5 Changing copy and content

- Public copy, statistics, board, focus areas and partnership routes: edit `config/phds.php`.
- Blog posts and campaign stories: use the back office.
- Campaign open dates and goals: back office.
- Seeded account passwords: edit `.env` and run `php artisan db:seed --class=UserSeeder --force`.

14. Testing

Run `php artisan test`. The suite passed 161 tests with 961 assertions at this commit.

Test file	Tests	What it covers
`PagesTest`	7	Public pages and the focus area routes
`ExampleTest` (Feature and Unit)	2	Placeholder tests from the starter kit
`BlogTest`, `BlogEditTest`	17	Blog CRUD, featured posts, picture and video rules
`VideoEmbedTest` (Unit)	13	YouTube and Vimeo link conversion and rejection
`ChatbotTest`	10	Assistant behaviour, limits and knowledge boundaries
`TeamChatTest`	29	Direct chats, groups, moderation, messages, rules
`EncryptedChatTest`	32	Key handling, rotation, idempotency, what the server may hold
`NotificationTest`	9	Notification creation, collapsing and delivery
`UserManagementTest`	12	Admin user CRUD, role protection
`UserSeederTest`	6	Seeding from `.env`, reseeding, password checks
`ProfileTest`	5	Profile update, password change and account deletion
Authentication (`tests/Feature/Auth`)	19	Login, registration, verification, password reset and confirmation

The end-to-end encryption primitives were also checked in Node with the real WebCrypto engine (13 checks), and the full encrypted chat flow was run in three real browsers against the running application. Those scripts are not in the repository (section 15).

Not covered by the suite: the donation flow. A donation test file was written and parked, because its fake Daraja responses were guesses rather than Safaricom's real behaviour. It should be rewritten after a sandbox payment has been observed (section 15).

15. Known issues and gaps

Listed in priority order.

1. **Successful M-Pesa payment not yet observed end to end.** The request, acknowledgement, callback delivery and status handling work. In sandbox, prompts to test numbers have returned result 1037, so the success branch is verified only through the code paths and the simulator, not a real payment. The Daraja simulator has to be used to complete one.
2. **Donation tests are missing.** Rewrite them against real sandbox responses, covering idempotency, every status transition and the callback handling.
3. **Staff can edit any blog post.** `BlogController::edit` and `update` check only the role, not authorship. The UI shows staff only their own posts, but the URL works for any post. Fix with a `BlogPolicy` that allows admins everything and staff only their own posts.
4. **No donation thank-you page with a receipt.** The donation page shows the result. A shareable confirmation route is planned but not built.
5. **Contact and newsletter do not send email.** Submissions are stored in `inquiries` only. Add a mailable and a notification to the team.
6. **The scheduler is not in the container.** See section 12.3.
7. **No automatic migrations on deploy.** See section 12.3.
8. **Reverb is untested in production.** Real-time was tested on the local server, which does not use nginx, TLS or the production host names.
9. **Encryption has not had an external security review.**
10. **Browser end-to-end scripts are not committed.** The three-browser chat test and the donation UI tests exist only in the working copies. Commit them under `tests/e2e`.
11. **Login has only Breeze's built-in throttle** and there is no two-factor authentication.

12. **Admin campaign and donation screens** have no feature tests yet.
13. **The Gemini assistant has not been load tested.** The guard rails were tested with 23 sample questions, not with adversarial traffic.
14. **Only one M-Pesa environment is configured at a time.** Switching from sandbox to production means changing the `MPESA_*` values, and Daraja production needs the Kenyan paybill or till and the organisation portal set-up described in Safaricom's documentation.
15. **Timezone is UTC.** Donation and chat times are shown in the viewer's local time, but reports and CSV exports are in UTC.
16. **Docker image is large and not optimised.** It rebuilds all dependencies on every change to application code.
17. **Trusted proxies are not configured in the repository.** Without `trustProxies`, links and callback addresses behind Render's HTTPS proxy can be generated as `http`. See section 12.4.

16. Roadmap

Suggested order for the next developer:

1. Fix the blog ownership gap (item 3).
2. Complete one sandbox donation through the Daraja simulator. Rewrite the donation tests against the real responses.
3. Add the scheduler and migrations to the deployment, and commit the deployment files.
4. Add email notifications for contact and newsletter submissions.
5. Build the donation thank-you and receipt page.
6. Commission a security review of the chat encryption and the M-Pesa callback handling.
7. Commit the browser end-to-end tests.
8. Move the public copy and board members into a database-backed admin screen, so non-developers can edit them.

17. Glossary

Term	Meaning
Daraja	Safaricom's developer API portal for M-Pesa
STK push, M-Pesa Express	A payment prompt sent to a phone, approved with the M-Pesa PIN
Shortcode	The business number that receives payments
Passkey	Secret issued by Safaricom, used to build the request password
CheckoutRequestID	Identifier Daraja gives each prompt, used to match the callback
ResultCode	Outcome in the callback. `0` is success.
Idempotency	A repeated request has the same effect as one request
Conversation key	The AES key for one chat, sealed separately for each member
Key version	Increases when a conversation's key is rotated
Security code	Short code derived from a public key, compared between people to detect key substitution
Moderator	The owner of a chat group, with rights to manage it
Reconcile	Asking Daraja about payments whose callback has not arrived

18. Handover checklist

- [ ] Obtain access to the GitHub repository, the Daraja account, the Gemini key, and the hosting account.
- [ ] Obtain the production `.env` values from a secure password manager. Do not share them in chat or email.
- [ ] Rotate any credential that has been shared in plain text during development, including the sandbox initiator password and passkey.
- [ ] Confirm the production shortcode, passkey and transaction type with the organisation, and whether the organisation has a Kenyan M-PESA organisation portal login.
- [ ] Confirm who receives donation notifications and who can run `user:role`.
- [ ] Run the test suite and confirm 161 passing tests.

- [ ] Deploy to staging first, run one sandbox donation, and check the callback reached the site.
- [ ] Set up the scheduler and confirm `donations:reconcile` runs.
- [ ] Take a backup of the production database before the first deployment that adds a migration.
- [ ] Record the name and contact of the previous developer for questions.